ChatGPT

数据库“不确定性函数”相关历史 Bug 汇总

典型问题案例

以下表格汇总了主流数据库系统中与非确定性（**不确定性**）函数相关的已公开 Bug 或问题报告，包括涉及函数、问题描述、官方链接及概念验证示例（PoC）。这些问题多与随机数、当前时间、UUID 等**每次调用结果不固定**的函数有关，常导致执行结果与预期不符、语义错误或计划错误等现象。

MySQL/MariaDB

DBMS 版本	函数	问题描述	官方链接/编号	PoC 示例
MySQL 5.6.x	`RAND()`	**HAVING 子句中使用 RAND() 导致错误结果**：在带有 `HAVING rand()*100 < 常数` 的查询中，如果表上存在索引，优化器可能错误地使用索引扫描，返回不满足 HAVING 条件的行 [1] [2]（即结果包含 `r>=20` 的行，应排除却未排除）。该问题在无索引或 MySQL 5.5 下不会出现，属 MySQL 5.6 优化器的回归错误。	MySQL Bug #69638 [1] [2]	建立两张数据相同但索引不同的表，对比执行 `SELECT tid, (RAND()*100) AS r FROM ... GROUP BY tid HAVING r<20` 的结果：有索引表返回了不应出现的 `r>=20` 行 [1] [2]。
MySQL 5.0.x/ 5.1.x 等	`RAND()`	**WHERE 条件中使用 RAND() 导致结果不可预期**：在 `WHERE id = ROUND(RAND()*... ,0)` 查询中，本应至多返回1行（按主键匹配），却可能返回0行、1行甚至多行。[3] [4] 原因是 RAND() 在WHERE条件对每行重复计算，导致不同行各自匹配不同的随机值。MySQL 未将该行为视为可修复Bug（“Won’t fix”），因为这是非确定性函数逐行求值的既有逻辑，只是容易令用户困惑。	MySQL Bug #39274 [3] [4]	创建含自增主键的数据表，多次执行 `SELECT id FROM tbl WHERE id = ROUND(RAND()*N,0)`：观察每次返回行数变化，有时返回空集，有时返回多行，主键值各不相同 [4]。
MySQL 4.x/5.x	`RAND()`	**GROUP BY 中多次计算 RAND() 导致错误**：在带聚合的 GROUP BY 查询中，`RAND()` 对同一行可能计算多次，导致分组键不一致，引发诸如 “Duplicate entry ... for key” 的错误 [5] [6]。MySQL手册旧版仅提醒了 ORDER BY RAND() 多次计算的行为，此案例拓展了 GROUP BY 下 RAND() 的不可靠性 [6]。此问题最终被认定为文档遗漏而非软件缺陷。	MySQL Bug #8652 [5] [6]	执行 `SELECT LEFT(RAND(),3), a, COUNT(*) FROM r1 GROUP BY 1`，预期按截断随机值分组。但实际执行报错：“Duplicate entry '0.5' for key 1” [7]（因同一行被归入不同组）。

DBMS 版本	函数	问题描述	官方链接/编号	PoC 示例
MySQL 5.0.x	`RAND()`	**复制（Replication）过程随机数重播错误**：在基于语句复制（SBR）下，如果**存储过程中多次调用 RAND()**，从库收到的 binlog 种子重复，导致除第一次外的随机值与主库不一致 [8] [9]。主库插入两行随机数（不同），从库却重现了相同的两行（第二行错误地重复了第一行值） [9]。此乃 MySQL 5.0 的已知限制：每个函数调用只能保证第一次 RAND() 正确重现 [10]。MySQL 5.1 引入行复制（RBR）后可解决该问题。	MySQL Bug #25543 [8] [9]	在主库创建存储过程：连续两次 `INSERT ... VALUES(RAND())` 后 SELECT结果，两行值不同 [11]；然后检查从库同表，发现第二行与第一行完全相同 [9]。查看binlog可见两次 RAND 使用了同一对种子 [12] [13]。

PostgreSQL

DBMS 版本	函数	问题描述	参考链接	说明/示例

PostgreSQL 9.x 等	`random()`（以及序列函数 `nextval()`）	**子查询中 volatile 函数的计算次数出人意料**：PostgreSQL 将函数按**稳定性**分类（VOLATILE/STABLE）。如果在**非相关子查询**中调用 `random()`，查询计划会将其视作常量，只执行一次，所有行返回相同值；但在子查询引用了外部变量（相关子查询）时，则会每行重新计算 [14] [15]。这导致类似 `SELECT (SELECT random()) FROM generate_series(1,10)` 返回10行完全相同的随机数 [16]（因为内部子查询不依赖外部，每次查询仅计算一次），而加一个无关外部的表达式即可使其每行不同 [17]。PostgreSQL 核心开发者认为这是执行器实现所致的**预期行为**，但容易让用户误以为 BUG。官方在文档中强调了相关/非相关子查询对VOLATILE函数的影响 [18]。	PostgreSQL 黑客邮件列表讨论 [14] [15]（2016年）	执行：
				`SELECT (SELECT random()) FROM generate_series(1,10);`
				得到10行相同随机值 [16]。
				再执行：
				`SELECT (SELECT random()+ i*0) FROM generate_series(1,10) AS i;`
				得到每行不同的随机值 [17]（`i*0` 引入相关性）。

注： PostgreSQL 对于诸如 `current_timestamp` / `now()` 这类函数，在单个查询中按标准定义返回**固定值**（即标记为 STABLE，每次查询调用结果相同）。因此这类时间函数在查询内不会多次计算，不会产生像 RAND() 那样的结果不一致问题。但需注意 `clock_timestamp()` 等每次调用都会变（VOLATILE）。目前未查询到 PostgreSQL 在此方面的严重逻辑错误报告。

Oracle

DBMS 版本	函数	问题描述	参考链接	说明/示例
Oracle 12c	`DBMS_RANDOM.VALUE` (PL/SQL 内置随机函数)	**多次出现的非确定性函数被当作一次计算**：Oracle在SQL语句中调用PL/SQL函数可能做优化。例如在一条SELECT里对 `DBMS_RANDOM.VALUE` 求平均值两次，Oracle并未各自独立计算随机序列，而是**每行仅生成一次随机数**并用于两个聚合，从而两个 AVG 结果**完全相同** [19]。正常情况下，两列独立随机的平均值严格相等的概率几乎为零，该现象表明优化器错误地将此非确定性函数当作可重复的稳定值处理了 [20] [21]。Oracle 官方认为这是由于SQL引擎对函数调用进行了投影合并所致，并未承诺每次出现都单独计算，因此不视为Bug（属于实现细节） [22] [23]。但从用户角度看，这种行为是**违背直觉的语义错误**。	Oracle AskTom 讨论/技术博客 [19] [21]	**示例1：** 查询 `SELECT AVG(DBMS_RANDOM.VALUE), AVG(DBMS_RANDOM.VALUE) FROM all_objects WHERE ROWNUM<50000;` 返回两列结果相等 [19]。 **示例2：** 若不使用聚合函数，直接 `SELECT DBMS_RANDOM.VALUE, DBMS_RANDOM.VALUE FROM ...` 则每行两值不同，符合预期 [24]。对比说明在聚合情况下Oracle进行了非常规的函数处理。

注： Oracle 的当前时间函数如 `SYSDATE`、`CURRENT_TIMESTAMP` 每次调用通常都会返回**执行时刻**（在单条SQL中多次调用可能有微秒级差异，Oracle允许 `SYSDATE` 非常短间隔内略增），但不会像上述随机函数那样出现跨行一致性的问题，因此未见相关Bug报告。

SQL Server

DBMS 版本	函数	问题描述	官方链接	PoC 示例
Microsoft SQL Server 2000/2005/2008	`NEWID()` (UUID/GUID 生成)	**视图/CTE 中使用 NEWID() 在连接时计算次数异常**：如果在**派生表或视图**中为每行生成一个 `NEWID()`（期望每源行一个GUID），再与另一表做1对多 JOIN，SQL Server 有时会因为执行计划选择，在**JOIN 的目标行级别重复计算 NEWID()** [25]。结果表现为：同一个源表行在连接结果中出现多次时，每次却得到不同的 NEWID 值，违背了“应对源行只生成一次 GUID”的直觉。[25] 这个BUG最早由 Thomas Glörfeld提出，后被 Itzik Ben-Gan 研究证实，在某些执行计划（如嵌套循环）下出现 [25]。微软 Connect 平台收录了此问题（现已归档）。	MS Connect #338410 (已归档)；Itzik Ben-Gan 博客 [25]	建两表 t1 (主表) 和 t2 (从表，多行可引用同一 t1.id)。建视图 v1：`SELECT id, NEWID() AS guid_val FROM t1`。查询: `SELECT a.id, b.id, a.guid_val FROM v1 AS a JOIN t2 AS b ON a.id=b.id;` 实际输出中，t1的一行连接出多行时，a.guid_val每次不同 [25]，证明NEWID()被重复计算了。预期应每个 a.id 只有一个 GUID，实际却随 b.id 改变。

SQLite

DBMS 版本	函数	问题描述	参考链接	说明/示例
SQLite 3.7.4 等	`random()`	**子查询内 random() 计算行为不一致**：SQLite 的查询优化在某些情况下也会导致非确定性函数的调用次数差异。例如： `SELECT a, a+1 FROM (SELECT random() & 65535 AS a);` 返回的第二列始终是第一列加1（说明子查询的 `random()` 只算一次，a+1沿用同一值）[26]。但如果稍作改写： `SELECT a, a+1 FROM (SELECT random() & 65535 AS a FROM (SELECT 'x'));`（引入一个多余嵌套子查询），返回结果中第二列不再是第一列加1 [27]——即 `random()` 被计算了两次，a 和 a+1 各得不同随机值 [27]。开发者指出这是优化器针对不确定性函数处理不同所致 [28]。此现象在SQLite当时版本中属于已知但未明确修复的问题（可能被认为不影响功能）。	SQLite 邮件列表讨论 [26] [28]	如左所述两个查询对比：第一种写法输出形如 `(39692, 39693)` [26]（a+1正好等于a），第二种输出如 `(32219, 10986)` [27]（a+1不再是a加1）。说明第二种情况下 `random()` 发生了两次评估。

MongoDB

DBMS 版本	功能/阶段	问题描述	官方链接	说明/示例
MongoDB 3.4–3.6	`$sample` 聚合阶段	**随机游标采样可能返回错误/失败**：MongoDB提供 `$sample` 管道从集合中随机抽取文档。但在3.x版本中，若集合存在**大量删除残留记录**或请求样本数较大，随机游标可能多次命中已删除或重复的记录。内部重试超过100次仍未找到新记录时，查询直接报错[29]：“`$sample stage could not find a non-duplicate document after 100 ...`”，错误码28799。[29] 这一 bug 最初报告为 SERVER-20385，在3.1.9曾尝试修复，但在3.4/3.6仍可重现，尤其在**集合碎片多**或**一次采样量接近集合大小**时更易触发。MongoDB后续版本优化了随机游标算法以避免此问题。	MongoDB Jira SERVER-20385 [29]（已修复）	使用一个有大量删除碎片的集合执行： `db.collection.aggregate([{$sample: {size: N}}])` 经多次尝试后可能抛出错误：[29] `"$sample stage could not find a non-duplicate document after 100 while using a random cursor..."`。

共性分析

综上，不同数据库的不确定性函数 Bug 虽各有表象，但呈现出**一些共性**：

- **函数非确定性引发优化困难**：这些函数每次调用结果不可预测，优化器或执行器在决定**调用次数**和**调用时机**时容易出错。许多问题源自引擎错误地**假定函数结果可重用**或**需要重复计算**，导致结果不一致。例如，MySQL 和 SQLite 的某些查询在有/无索引或不同子查询结构下，`RAND()` / `random()` 的计算次数

不同，出现逻辑差异；SQL Server 则在连接优化中意外多算了 NEWID() [25]；Oracle 聚合中则少算了随机函数 [19]。可见，**如何在执行计划中正确对待Volatile函数**是共同难点。

- **结果表现为逻辑层错误**：与典型的崩溃或越界漏洞不同，这类Bug主要造成**查询语义上的偏差**：返回多余或遗漏行、输出重复/错误的值，或者如MongoDB `$sample` 直接报运行错误 [29]。这些错误往往不明显触发异常，但违背了SQL语义或用户预期（如违反逻辑的结果集、不一致的复制数据等），属于**逻辑正确性漏洞**。

- **有的被视为设计限制**：值得注意的是，部分问题（如MySQL的逐行RAND、多次RAND复制问题 [10]、PostgreSQL的子查询行为）被官方定位为已知限制或文档问题，而非可修复的bug。这体现出数据库对不确定性函数**行为定义的模糊地带**：为了性能或实现方便，某些引擎选择牺牲严格的逐调用独立性。例如Oracle和SQLite案例中官方均倾向认为“如此运作符合我们的实现逻辑”，用户则可能感觉“违背直觉”。因此这类问题有时难以严格归类为Bug，反而以**注意事项**形式记录。

- **触发往往有特定条件**：这些Bug通常在**复杂查询或特定环境**下才出现：比如**结合索引/优化器决策**（MySQL #69638）、**多表连接**（SQL Server NEWID bug）、**存储过程和复制**（MySQL #25543）、**数据分布异常**（MongoDB 大量删除碎片）等。简单场景往往难以察觉问题，只有在特定边界条件或大规模数据下才暴露。这也增加了发现的难度。

检测难点与潜在策略

检测难点： 非确定性函数Bug的测试和发现十分具有挑战，原因在于：

- **结果不可预测**：随机数或当前时间每次不同，传统的**固定预期值**测试无效。一次查询结果看似正常，可能无法直接判定错误，只有多次执行结果对比或特殊分析才能发现异常。例如MySQL RAND()多行问题需多次运行对比行数变化 [4]；PostgreSQL随机函数问题需要构造相关/非相关子查询两种场景比对。

- **覆盖组合复杂**：这类Bug往往触发于优化器特定决策或特定语法组合，**穷举测试空间巨大**。优化器选择受统计信息、索引、有无子查询等影响，很难手工覆盖所有情况。有的Bug如SQL Server NEWID，需要特定的1对多连接执行计划才出现；MongoDB `$sample` 错误则要特定数据分布。检测框架若不智能探索，很可能漏掉这些边角情况。

- **区分预期与异常困难**：部分“不一致”行为在官方定义里可能被视为正常（例如PostgreSQL/Oracle的情况），测试工具需了解各DBMS对不确定性函数的**官方契约**，否则可能误报。反之，真正的逻辑Bug有时隐藏在结果随机性中，不易与统计噪声区分，需要大量重复试验提高信心。

潜在检测策略： 针对上述难点，可以采用以下策略提高发现此类问题的可能性：

- **元操作测试（Metamorphic Testing）**：这是检测不确定性函数错误的有力方法 [18]。核心思想是构造多个逻辑等价、但函数调用时机不同的查询，比较它们结果是否一致。例如：
- **恒值替换**：先独立获取一次函数值，用这个常量替代函数再查询，比较与直接用函数的差异。对于应只调用一次的场景，两者结果应一致，否则说明函数被重复计算或共享。如MySQL `WHERE id = RAND()` 用变量保存 RAND 再查询对比 [30]。
- **查询重写**：像PostgreSQL例子，将子查询变相关或非相关，观察结果集合是否符合预期关系；或对同一查询强制不同执行计划（比如提示不走索引）对比结果，检测是否一致（MySQL #69638 就是索引与全表扫描结果应相同 [2]）。

- **多次执行比较**：对纯随机输出，如 `$sample`，可多次运行统计结果分布或错误出现频率，判断是否有异常偏差或报错概率过高来指示问题。

- **引入确定性模式**：为测试目的，可以修改或模拟不确定性函数为可控输出。例如在测试版内置一个**可预测的RAND(seed)**，让每次调用产生相同序列，这样在不同执行路径下输出可直接比对。若不同执行策略导致输出不一致，就能锁定问题（类似MySQL binlog对RAND种子的一致性检测）。

- **强化不确定性函数的标记和检查**：数据库开发中，可增加对VOLATILE函数使用的分析，比如：

- 在优化阶段检测如果试图将 volatile 表达式多个引用合并或下推时给出警告或禁止（防止Oracle那种情况）。

- 执行时加入断言验证：同一查询内多次出现的同一volatile函数如果按计划应独立计算，但结果意外相同/不同，则记录报警。

- **利用模糊测试框架**：借助诸如 SQLancer 等数据库模糊测试工具，生成大量随机查询，其中包含随机函数、当前时间等，并自动验证数据库的一些**不变性**。例如交叉比对开启/关闭某优化选项时结果是否一致，主从复制前后数据是否一致等，发现违反不变性的情况。这类工具已成功找到过许多逻辑错误，适用于不确定性函数这类棘手场景。

总之，不确定性函数相关Bug往往藏得很深，但通过**差分对比**、**约束重放**等策略可以提高检测成功率。一方面应完善数据库内对这类函数的优化规则约束，另一方面借助智能测试充分探索边界情况，及时发现并修复此类影响查询正确性的潜在漏洞。

参考资料：

1. MySQL Bugs: #69638 – Wrong results with RAND() in HAVING clause on indexed table [1] [2]
2. MySQL Bugs: #39274 – SELECT on PK with ROUND(RAND()) gives wrong rows [3] [4]
3. MySQL Bugs: #8652 – RAND() in GROUP BY with aggregate causes duplicate key error [5] [6]
4. MySQL Bugs: #25543 – RAND() in stored procedure replicates inconsistently (binlog) [8] [9]
5. PostgreSQL Hackers Mailing List – Strange behavior of volatile functions (random(), nextval()) in subqueries [14] [15]
6. Martin Widlake’s Oracle Blog – PL/SQL bug with DBMS_RANDOM? [19] [21]
7. Itzik Ben-Gan, SQL Server Magazine – Bug with NEWID Function in table expressions [25]
8. SQLite Mailing List – random() function does not keep value in subquery [26] [28]
9. MongoDB Jira SERVER-20385 – Error in $sample with random cursor (Location28799) [29]

---

1 2 MySQL Bugs: #69638: Wrong results when running a SELECT that includes a HAVING based on a function
https://bugs.mysql.com/bug.php?id=69638

3 4 MySQL Bugs: #39274: SELECT on PK with ROUND(RAND()) give wrong errors
https://bugs.mysql.com/bug.php?id=39274

5 6 7 MySQL Bugs: #8652: group by part of rand() returns duplicate key error
https://bugs.mysql.com/bug.php?id=8652

8 9 10 11 12 13 MySQL Bugs: #25543: Replication of wrong values if using rand() in stored procedure
https://bugs.mysql.com/bug.php?id=25543

14 15 18 PostgreSQL: Strange behavior of some volatile function like random(), nextval()
https://www.postgresql.org/message-id/flat/414eda7d-739e-0fea-cb50-2ab7c1c0152f%40postgrespro.ru

16 17 PostgreSQL: Strange behavior of some volatile function like random(), nextval()
https://www.postgresql.org/message-id/414eda7d-739e-0fea-cb50-2ab7c1c0152f%40postgrespro.ru

19 20 21 22 23 24 PL/SQL bug with DBMS_RANDOM? | Martin Widlake's Yet Another Oracle Blog
https://mwidlake.wordpress.com/2015/10/08/plsql-bug-with-dbms_random/

25 Bug with NEWID Function
https://www.itprotoday.com/early-versions/bug-with-newid-function

26 27 28 Re: [sqlite] random() function does not keep value in subquery
https://www.mail-archive.com/sqlite-users@mailinglists.sqlite.org/msg57688.html

29 BugZero | MongoDB BugID 390209 - $sample stage could not find a non-duplicate docum...
https://www.findbugzero.com/operational-defect-database/vendors/mongodb/defects/390209

30 Nondeterministic Functions in MySQL (i.e. rand) Can Surprise You
https://www.percona.com/blog/nondeterministic-functions-in-mysql-i-e-rand-can-surprise-you/